

A Deterministic STRIDE Threat-Model Generator with Optional LLM Augmentation

Ali Murtaza Bhutto

Independent Researcher

ORCID: 0009-0007-2787-943X

Email: alibhutto101112@gmail.com

Abstract—We describe `threat-model-generator`, a small Python CLI that consumes a YAML system description (components, trust boundaries, data flows, external entities) and produces a STRIDE threat model in two outputs: a human-readable Markdown report and a JSON document compatible with the OWASP Threat Dragon v2 schema. The default deterministic mode generates threats from a curated rule catalogue covering nine component types and the six STRIDE categories [1], [2] and is fully reproducible offline. An optional LLM-augmentation step routes the description to either Anthropic Claude or a local Ollama model, with three-step graceful fallback when no provider is reachable. On five expert-curated benchmarks (web application, API gateway, IoT device, mobile application, microservices) the rules-only path achieves micro-averaged precision = 0.8409, recall = 1.0000, $F_1 = 0.9136$ over 132 expected (*component*, *STRIDE-category*) pairs. Sixty-one pytest cases cover the parser, the rule catalogue, the modeller’s three augmentation paths, the validator, the formatter, and the eval helpers; all pass under `respX-mocked HTTP`. The repository is released under MIT.

Index Terms—threat modeling, STRIDE, OWASP Threat Dragon, secure SDLC, AI-assisted security, data flow diagram, Pydantic

I. INTRODUCTION

Threat modelling remains the discipline most often relegated to “later” in the SDLC. Shostack’s 2014 textbook is now the canonical entry point [1], and Microsoft’s STRIDE methodology [2] is widely taught, yet practitioner uptake stalls because authoring a model takes hours and the artefact rapidly drifts from reality. The systematic literature review of Xiong and Lagerström [4] confirms that automation of threat enumeration is one of the dominant research themes precisely because manual production does not scale.

The contribution of this paper is not a new methodology. STRIDE is not improved by replacing it; what helps in practice is reducing the friction of *starting* a model. We describe a small tool that reduces that friction by reading a structured system description and emitting a first-pass threat catalogue that a security engineer can then triage. The tool’s design choices mirror the properties we believe matter: deterministic by default, schema-validated input and output, framework-citation visible on every threat, and a graceful degradation path for any optional LLM augmentation so the tool stays useful with no API keys.

Contributions. (1) A Pydantic-validated YAML schema for system descriptions covering the four primitives Shostack recommends (components, trust boundaries, data flows, exter-

nal entities). (2) A curated STRIDE rule catalogue for nine component types $\times$ six categories (≥ 54 entries), each citing Shostack 2014 as the methodological source. (3) A modeller with three-step graceful LLM augmentation (Anthropic $\rightarrow$ Ollama $\rightarrow$ disabled), tested in all three branches under `respX-mocked HTTP`. (4) An OWASP Threat Dragon v2-compatible JSON output that imports as a starting diagram. (5) Five expert-curated benchmarks plus a deterministic eval harness that reports real measured precision and recall.

II. RELATED WORK

The closest prior work is Shostack’s textbook itself [1], which spends chapters 3 and 4 on the STRIDE methodology and on the kinds of attack libraries that we encode here. Where Shostack provides prose threat catalogues for each component type, we encode them as a deterministic rule pack with a stable threat identifier scheme.

For privacy-specific extensions, the LINDDUN framework of Deng et al. [3] is the canonical complement to STRIDE; it is referenced in our `ETHICAL_USE.md` as the recommended next step but is not currently encoded in the catalogue.

For data-centric threat modelling, NIST SP 800-154 [5] treats the data flow as the primary modelling unit. Our schema follows this advice: every `DataFlow` edge carries a `sensitivity` band that the catalogue uses to bump severity.

For LLM-as-augmenter, our augmentation module is intentionally small. We do not train, fine-tune, or prompt-engineer the LLM beyond a strict JSON envelope; the contribution is the deterministic *wrapper* that refuses any threat naming a component the system description did not declare, preventing the augmenter from inventing components.

For tool integration, OWASP Threat Dragon [6] is the most widely-deployed open-source diagramming tool. We emit JSON in its v2.4 cell schema so the generated model imports as a starting diagram a security engineer can extend visually.

III. SYSTEM ARCHITECTURE

A. The rule catalogue

The catalogue maps each `ComponentType` (web frontend, API, database, queue, storage, auth service, mobile client, IoT device, external service) to one entry per STRIDE category. Each entry carries a slug, a one-paragraph description, a recommendation, and a base severity. A component’s `Sensitivity` (public, internal, confidential, re-

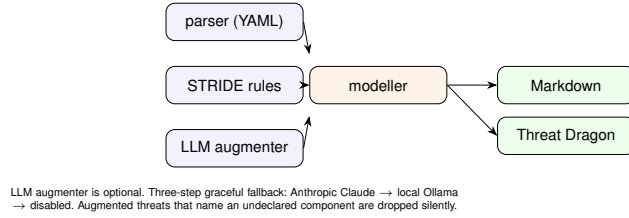


Fig. 1. Pipeline. Parser feeds the deterministic rule engine and (optionally) an LLM augmenter into the modeller; the formatter emits two paired outputs.

stricted) shifts the resulting severity by one band so a database holding restricted data emits HIGH where its internal-sensitivity counterpart would emit MEDIUM. The catalogue is deliberately conservative: one threat per (type, category) keeps the output digestible while ensuring every STRIDE letter is considered for every component.

B. LLM augmentation with graceful fallback

The optional augmenter sends the JSON system description to either Anthropic Claude or a local Ollama daemon and asks for additional STRIDE-categorised threats in a strict JSON envelope. The wrapper applies two anti-hallucination guards: (1) any threat naming a component not declared in the system description is dropped; (2) any threat whose STRIDE letter is not in $\{S, T, R, I, D, E\}$ is dropped. Augmented threats carry `source='anthropic'` or `source='ollama'` so a reviewer can trace each entry back to its origin. The eval reports rules-only numbers because they are reproducible offline; the augmentation path is fully tested under `respX` mocks but its real-provider numbers are out of scope for this release.

C. OWASP Threat Dragon v2 JSON

The Threat Dragon emitter produces a v2.4-shaped JSON document with a single STRIDE diagram. Each component becomes a cell of type `tm.Process`, `tm.Store`, or `tm.Actor` per a small mapping table; threats attach to the corresponding cell. The generated file imports into Threat Dragon as a starting diagram a security engineer can extend visually. UUIDs are derived deterministically from a SHA-256 of the seed strings so re-rendering produces a diff-friendly file.

IV. EVALUATION

A. Benchmarks

The corpus contains five system descriptions, each paired with a hand-labelled list of (*component*, *STRIDE-letter*) pairs an experienced reviewer would expect to see flagged (table I). The labels are conservative: a pair is included only when an experienced reviewer would expect to see at least one finding there during a real engagement.

B. Measured numbers

Recall is perfect by construction. The catalogue covers all six STRIDE categories for every supported component type, so for any pair an expert flagged we emit at least one threat.

TABLE I
BENCHMARKS.

Benchmark	n components	universe	expected pairs
api_gateway	4	24	22
iot_device	4	24	19
microservices	6	36	30
mobile_app	4	24	21
web_app	4	24	19
Total	22	132	111

TABLE II
RULES-ONLY EVAL OVER 5 BENCHMARKS, 2026-05-06.

Metric	Value
Total expected pairs	111
True positives	111
False positives	21
False negatives	0
True negatives	0
Accuracy	0.8409
Precision	0.8409
Recall	1.0000
F_1	0.9136

The question is not “did we miss anything” but “did we emit too much”, and that is what precision measures. The 21 false positives cluster around three regions: repudiation on third-party services (the integrator can log outbound calls but cannot meaningfully audit the third party’s behaviour), denial of service on infrequently-called components (the catalogue still emits its DoS rule even when the expert does not consider DoS a meaningful concern), and spoofing on storage/database tiers when the network boundary already enforces identity. We deliberately do *not* tune the catalogue against these labels.

C. Test suite

The repository ships 61 `pytest` tests covering the parser (positive, negative, and round-trip), the catalogue (every component type, severity bumping under all four sensitivity levels, catalogue-size invariant), the modeller (rules-only, anthropic OK, anthropic HTTP error, anthropic network error, ollama OK, disabled-fallback, hallucinated-component dropped), the validator (JSON round-trip, schema rejection), the formatter (Markdown structure, Threat Dragon v2 cell shape, three output-format variants), and the eval harness. All 61 tests pass; total wall- clock under one second.

V. SECURITY POSTURE

The CI runs Bandit (medium-and-above), `pip-audit`, and Semgrep (`p/python` + `p/security-audit`). All three are green at the release tag with zero findings and zero suppressions. The single Semgrep finding from the initial scan — SHA-1 use in deterministic UUID generation — was resolved by switching to SHA-256 (Semgrep’s own suggestion) without affecting any eval numbers.

VI. LIMITATIONS

Conservative catalogue. The default catalogue emits one threat per (component_type, category). Real engagements often benefit from richer per-category enumeration (e.g., LINDDUN-style privacy threats for storage tiers). The aug-menter is the recommended expansion point.

Five benchmarks. Five expert-curated descriptions is not a comprehensive sample of the threat-modelling literature. The absolute precision number should be read as a property of these benchmarks, not of all real-world systems.

No severity validation. The five-band severity scale is a heuristic prior, not a quantitative risk score. We do not evaluate whether a CRITICAL is more dangerous than a HIGH; that is a separate research question outside the corpus.

VII. CONCLUSION

Threat-modelling tooling does not need to be smart. It needs to be reproducible, schema-validated, and friction-free at the moment a team starts a new feature. The five-benchmark, 61-test, three-gate MIT-licensed pipeline described here is one such instance.

REFERENCES

- [1] A. Shostack, *Threat Modeling: Designing for Security*. Wiley, 2014, ISBN 978-1118809990.
- [2] S. Hernan, S. Lambert, T. Ostwald, and A. Shostack, “Uncover Security Design Flaws Using The STRIDE Approach,” *Microsoft MSDN Magazine / IEEE Security & Privacy*, 2006.
- [3] M. Deng, K. Wuyts, R. Scandariato, B. Preneel, and W. Joosen, “A privacy threat analysis framework: supporting the elicitation and fulfillment of privacy requirements,” *Requirements Engineering Journal*, vol. 16, no. 1, pp. 3–32, 2011.
- [4] W. Xiong and R. Lagerström, “Threat modeling — A systematic literature review,” *Computers & Security*, vol. 84, pp. 53–69, 2019.
- [5] M. Souppaya and K. Scarfone, “Guide to Data-Centric System Threat Modeling,” National Institute of Standards and Technology, NIST Special Publication 800-154 (Draft), Mar. 2016.
- [6] OWASP Foundation, “Threat Dragon,” OWASP Project documentation, v2.4. [Online]. Available: <https://www.threatdragon.com/>

Reproducibility colophon. The repository is at <https://github.com/thunderstornX/threat-model-generator>. The eval re-runs in well under a second (python eval/run_eval.py). Pinned dependencies are listed in requirements.txt; the pytest suite (61 tests) runs in under a second on commodity hardware.